

AeroHexMesh: Mathematical Formulations

A companion to `docs/` – every algorithm’s math, in one document

Contents

Scope	1
1 Construct2D/	1
2 Construct2D_to_SOD2D/linear_mesh/	1
2.1 Spanwise extrusion	1
2.2 Structured node numbering	2
2.3 Inlet/outlet classification by angle of attack	2
2.4 Wall-spline fit: natural cubic spline	2
2.5 Gmsh periodicity	3
3 Construct2D_to_SOD2D/high_order_mesh/	3
3.1 Parametric (arc-length) high-order wall placement	3
3.2 Linear elasticity relaxation	3
4 Construct2D_to_NEKRS/linear_mesh/	4
4.1 Format-conversion chain (<code>arglist.sh</code>)	4
4.2 Case-parameter extraction from the NMF	4
4.3 Fischer’s natural cubic spline (<code>spline/splint</code>)	4
4.4 Weighted-Laplace boundary blending	4
5 Construct2D_to_NEKRS/high_order_mesh/	5
6 pyHyp/	5
6.1 Surface coarsening	5
6.2 Local surface curvature	5
6.3 Curvature monitor field	6
6.4 Curvature-weighted 1D equidistribution	6
6.5 Weighted elliptic relaxation + Newton reprojection	6
7 etaGrid_to_SOD2D/	6
7.1 Airfoil curve: fit and differential geometry	7
7.2 Vinokur (1983) two-sided stretching	7
7.3 Body-wrap sweep geometry	7
7.4 Wake growth: geometric offset series	8
7.5 Boundary classification (local coordinates)	8
7.6 Wavy-wall perturbation	8

7.7 Wall-node arc-length recovery (verification)	9
8 Cross-cutting: boundary conditions	9

Scope

This document collects, in standard mathematical notation, every algorithm documented (in prose + pseudocode) across `docs/`. It is organized identically to that tree: one section per pipeline directory, in the same order. Each section cross-references the corresponding Markdown page, which additionally covers *where* each piece of math lives in code and *why* each design decision was made – this document is the math alone, typeset properly.

Notation used throughout: bold lowercase ($\mathbf{p}$, $\mathbf{n}$) for 2D or 3D vectors/points; s for an arc-length parameter; subscripts for grid/element indices; primes or ∂ for derivatives as is conventional per context.

1 Construct2D/

Construct2D’s own grid-generation PDE (elliptic or hyperbolic, selected by the `slvr` namelist field) is vendored upstream Fortran and out of scope here – see `Construct2D/doc/user_manual.pdf`. No original math from this repository’s own code is introduced in this directory; see `docs/Construct2D/README.md` for the batch-driving mechanism.

2 Construct2D_to_SOD2D/linear_mesh/

See `docs/Construct2D_to_SOD2D/linear_mesh.md`.

2.1 Spanwise extrusion

Given a 2D structured grid (x_{ij}, y_{ij}) , $i \in [0, \text{idim}), j \in [0, \text{jdim})$, and spanwise stations $z_k = k \cdot \Delta z$, $k \in [0, \text{nplanes})$, extrusion is pure translation:

$$X_{ijk} = x_{ij}, \quad Y_{ijk} = y_{ij}, \quad Z_{ijk} = z_k. \quad (1)$$

No offset or curvature is introduced – contrast with Section 7.3’s curved sweep.

2.2 Structured node numbering

For block n with per-block node count $N_n = \text{idim}_n \cdot \text{jdim}_n \cdot \text{kdim}_n$, the flat (1-based) node id of structured index (i, j, k) is

$$\text{id}(n, i, j, k) = 1 + \underbrace{\sum_{m < n} N_m}_{\text{base}_n} + k + d_k j + d_k d_j i, \quad (2)$$

where d_j, d_k are block n ’s own J/K extents (K fastest-varying, Fortran-style). Hexahedra are built by walking every interior (i, j, k) cell and gathering its 8 corners via a fixed $(-1, 0)$ -offset shift table in each axis.

O-grid periodic closure. The last i -plane ($i = \text{idim} - 1$) is never stored; any reference to it is rewritten as $i \leftarrow 0$ before evaluating $\text{id}(\cdot)$ – an *aliasing* closure, not a duplicated node with a periodic link.

C-grid wake-cut fold. A sub-range of the wall row ($j = 0$) is aliased onto another sub-range of the *same* row via a constant index offset:

$$i_B = \omega - i_A, \quad \omega = s_{2,B} - s_{2,A}, \quad (3)$$

where $s_{2,A}, s_{2,B}$ are the two matched NMF sub-range start indices. Side B 's nodes are never stored; every reference to an index in $[i_{\text{lo},B}, i_{\text{hi},B}]$ at $j = 0$ is rewritten through this map first.

2.3 Inlet/outlet classification by angle of attack

At each farfield boundary point (i, k) , with wall-row point $\mathbf{p}_w = (x_{i,0,k}, y_{i,0,k})$ and farfield-row point $\mathbf{p}_f = (x_{i,\text{jmax},k}, y_{i,\text{jmax},k})$:

$$\hat{\mathbf{d}} = \frac{\mathbf{p}_f - \mathbf{p}_w}{\|\mathbf{p}_f - \mathbf{p}_w\|} \quad (\text{local outward direction}) \quad (4)$$

$$\mathbf{U}_\infty = (\cos \alpha, \sin \alpha) \quad (\text{free-stream direction, } \alpha = \text{AoA, mesh built at } \alpha = 0) \quad (5)$$

$$\text{class} = \begin{cases} \text{inlet,} & \hat{\mathbf{d}} \cdot \mathbf{U}_\infty < 0 \quad (\text{flow enters}) \\ \text{outlet,} & \hat{\mathbf{d}} \cdot \mathbf{U}_\infty \geq 0 \quad (\text{flow leaves}) \end{cases} \quad (6)$$

For a C-grid, the two wake-end faces are always classified outlet, bypassing this test entirely (their own outward direction is not meaningfully wall→farfield). **This sign convention has not been independently verified** – see [docs/reference/troubleshooting.md](#).

2.4 Wall-spline fit: natural cubic spline

Given ordered wall corner points (x_i, y_i) , the arc-length parametrization is cumulative chord length,

$$s_0 = 0, \quad s_i = s_{i-1} + \|\mathbf{p}_i - \mathbf{p}_{i-1}\|. \quad (7)$$

An open natural cubic spline is fit to $y(s)$ (and independently to $x(s)$); on segment i , for $s_i \leq t \leq s_{i+1}$, $\delta t = t - s_i$:

$$y(t) = a_i + b_i \delta t + c_i \delta t^2 + d_i \delta t^3. \quad (8)$$

The second-derivative values $M_i = y''(s_i)$ solve the classic tridiagonal system with natural boundary conditions $M_0 = M_n = 0$:

$$h_i = s_{i+1} - s_i, \quad h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6 \left[\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right], \quad (9)$$

solved by the Thomas algorithm (tridiagonal Gaussian elimination: forward sweep then back substitution). The per-segment coefficients follow directly:

$$a_i = y_i, \quad b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i(2M_i + M_{i+1})}{6}, \quad (10)$$

$$c_i = \frac{M_i}{2}, \quad d_i = \frac{M_{i+1} - M_i}{6h_i}. \quad (11)$$

2.5 Gmsh periodicity

The spanwise periodic pair is declared as a pure translation:

$$\text{Periodic Surface } \{k_{\max}\} = \{k_0\} \text{ Translate } (0, 0, \text{span}_z). \quad (12)$$

3 Construct2D_to_SOD2D/high_order_mesh/

See [docs/Construct2D_to_SOD2D/high_order_mesh.md](#).

3.1 Parametric (arc-length) high-order wall placement

For a high-order wall node with reference (Gauss–Lobatto–Legendre) coordinate $\xi_k \in [-1, 1]$ along a face’s own along-wall axis, between two corners matched to spline arc-lengths $s_{\text{lo}}, s_{\text{hi}}$ (segment index $\text{iseg} = \min$):

$$s_f = s_{\text{lo}} + (s_{\text{hi}} - s_{\text{lo}}) \cdot \frac{1}{2}(\xi_k + 1), \quad (13)$$

$$\delta t = s_f - s_{\text{iseg}}, \quad (14)$$

$$x_t = a_x + \delta t (b_x + \delta t (c_x + \delta t d_x)), \quad (15)$$

$$y_t = a_y + \delta t (b_y + \delta t (c_y + \delta t d_y)), \quad (16)$$

$$\mathbf{u}_{\text{buffer}} = \lambda_{\text{scale}} \cdot ((x_t, y_t) - (x_{\text{current}}, y_{\text{current}})), \quad (17)$$

where $(a, b, c, d)_{x,y}$ are the segment’s cubic coefficients from Section 2.4 and λ_{scale} is an optional damping factor (default 1, no damping). This is a *parametric*, not nearest-point, placement – deliberately, since a Euclidean nearest-point search is a many-to-one map near high curvature.

3.2 Linear elasticity relaxation

The imposed wall displacement becomes a Dirichlet condition; the interior solves the isotropic linear elasticity equilibrium equation

$$\nabla \cdot \boldsymbol{\sigma} = \mathbf{0}, \quad \boldsymbol{\sigma} = \lambda(\nabla \cdot \mathbf{u}) \mathbf{I} + 2\mu \boldsymbol{\varepsilon}(\mathbf{u}), \quad \boldsymbol{\varepsilon}(\mathbf{u}) = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^\top), \quad (18)$$

with the Lamé parameters computed directly from Young’s modulus E and Poisson’s ratio ν :

$$\mu = \frac{E}{2(1 + \nu)}, \quad \lambda = \frac{E\nu}{(1 + \nu)(1 - 2\nu)}. \quad (19)$$

Discretized with the same spectral-element basis the flow solver itself uses, solved by preconditioned conjugate gradient (block-Jacobi or scalar-Laplacian-diagonal preconditioner). After solving, if the resulting quality $\min Q < 10^{-6}$ (an inverted/degenerate element), the solver reverts and performs a small grid search over (E, ν) (coarse 2-point, then finer 9-point), re-solving each time and keeping whichever combination maximizes $\min Q$.

4 Construct2D_to_NEKRS/linear_mesh/

See [docs/Construct2D_to_NEKRS/linear_mesh.md](#). Extrusion/NMF-remap/node-numbering/AoA-classification math is shared with Section 2–2.3. This section covers what is genuinely different: Nek5000’s own conversion chain and Fischer’s independently-implemented smoothing algorithm.

4.1 Format-conversion chain (`arglist.sh`)

The spanwise extrusion itself happens *inside* Nek5000’s own tools, not in Python:

$$.\text{re2} \xrightarrow{\text{re2toarea}} .\text{rea} \text{ (2D)} \xrightarrow{\text{n2to3}} .\text{rea} \text{ (3D, extruded+periodic)} \xrightarrow{\text{reatore2}} .\text{re2} \text{ (3D)}. \quad (20)$$

4.2 Case-parameter extraction from the NMF

Reading the native 2D NMF’s own `VISCOUS` entry ($f_1 = 3$, excluding the C-grid wake-cut `ONE_TO_ONE` pairing):

$$\text{ring_width} = \text{idim} - 1, \quad \text{radial_rings} = \text{jmax} - 1, \quad (21)$$

$$\text{wall_start_element} = s_1^{\text{VISCOUS}}, \quad \text{wall_element_count} = e_1^{\text{VISCOUS}} - s_1^{\text{VISCOUS}}. \quad (22)$$

4.3 Fischer’s natural cubic spline (`spline/splint`)

Independently implemented (Fortran77, Numerical Recipes style) but mathematically the *same* construction as Section 2.4: natural boundary conditions, tridiagonal solve for y'' . Evaluation (`splint`) uses the equivalent cubic-Hermite-style blend, given bracketing points $x_{\text{lo}}, x_{\text{hi}}$ ($h = x_{\text{hi}} - x_{\text{lo}}$):

$$a = \frac{x_{\text{hi}} - x}{h}, \quad b = \frac{x - x_{\text{lo}}}{h}, \quad (23)$$

$$y = a y_{\text{lo}} + b y_{\text{hi}} + [(a^3 - a) y''_{\text{lo}} + (b^3 - b) y''_{\text{hi}}] \frac{h^2}{6}. \quad (24)$$

Every high-order node is placed at its own GLL-reference arc-length fraction along its segment,

$$s_f(j, e) = s_{i-1} + \frac{1}{2} \text{ds}(\zeta_j + 1), \quad \zeta_j = \text{the GLL reference coordinate} \in [-1, 1], \quad (25)$$

the same conceptual move as Section 3.1 – independently arrived at in a separate codebase.

4.4 Weighted-Laplace boundary blending

Given the wall’s own displacement field δu (from Section 4.3), extend it into the volume by solving a *weighted Laplace* equation (not the full elasticity tensor of Section 3.2):

$$\nabla \cdot (h_1(\mathbf{x}) \nabla u) = 0, \quad u = \delta u \text{ on the wall}. \quad (26)$$

Boundary-layer protection via a spatially-varying diffusion coefficient:

$$\delta = \frac{\text{volume}(\text{wall-adjacent elements})}{\text{surface area}(\text{wall-adjacent elements})} \quad (\text{avg. boundary-layer element thickness}) \quad (27)$$

$$\delta_p = 5\delta \quad (\text{“protected” thickness}) \quad (28)$$

$$h_1(\mathbf{x}) = 1 + 9 \exp\left(-\left(\frac{d(\mathbf{x})}{\delta_p}\right)^2\right) \quad (d = \text{distance to wall}; h_1 \rightarrow 10 \text{ at the wall, } \rightarrow 1 \text{ far away}) \quad (29)$$

Solved via the standard Dirichlet-lift decomposition $u = u_0 + u_b$ (u_b = boundary data extended by duplicate-node averaging; $u_0 = 0$ on Dirichlet boundaries):

$$\bar{A} u_0 = -M \bar{A} u_b, \quad (30)$$

where $\bar{A}$ is the (weighted) Laplace/Helmholtz operator and M the interior/Dirichlet mask; x - and y -displacement are solved as two independent scalar problems (no shear/Poisson coupling between components, unlike Section 3.2).

5 Construct2D_to_NEKRS/high_order_mesh/

See [docs/Construct2D_to_NEKRS/high_order_mesh.md](#). No new smoothing math: this step *interpolates* Section 4’s already-corrected, fixed-order ($\ell_{x_1} = 4$) reference sampling onto the production mesh’s own (possibly different) polynomial order via a standard Lagrange-basis change-of-interpolant operation (`map_m_to_n`: an m -point to n -point GLL resampling), then copies that corrected (x, y) identically to every spanwise plane (pure-translation extrusion, so no recomputation needed there):

$$x_{m1}(i, j, i_z, e) = x_{m1}(i, j, 1, e) \quad \forall i_z = 2, \dots, \ell_{z1}. \quad (31)$$

6 pyHyp/

See [docs/pyHyp/README.md](#). pyHyp’s own hyperbolic marching PDE is an external tool, out of scope. This section covers this repo’s own surface-preprocessing math.

6.1 Surface coarsening

Keep every other structured-grid index, plus the last index unconditionally (so both endpoints of an axis are always retained):

$$\text{keep}(n) = \{0, 2, 4, \dots\} \cup \{n - 1\}. \quad (32)$$

6.2 Local surface curvature

Given a per-block bicubic Hermite surface fit $\mathbf{P}(s, t)$ (evaluated per-cell, finite-difference step ε):

$$\mathbf{P}_s = \frac{\partial \mathbf{P}}{\partial s}, \quad \mathbf{P}_t = \frac{\partial \mathbf{P}}{\partial t} \quad (\text{central difference}) \quad (33)$$

$$\mathbf{P}_{ss}, \mathbf{P}_{tt}, \mathbf{P}_{st} \quad (\text{central-difference 2nd partials}) \quad (34)$$

$$\hat{\mathbf{n}} = \frac{\mathbf{P}_s \times \mathbf{P}_t}{\|\mathbf{P}_s \times \mathbf{P}_t\|} \quad (\text{unit normal}) \quad (35)$$

First fundamental form E, F, G and second fundamental form L, M, N :

$$E = \mathbf{P}_s \cdot \mathbf{P}_s, \quad F = \mathbf{P}_s \cdot \mathbf{P}_t, \quad G = \mathbf{P}_t \cdot \mathbf{P}_t; \quad L = \mathbf{P}_{ss} \cdot \hat{\mathbf{n}}, \quad M = \mathbf{P}_{st} \cdot \hat{\mathbf{n}}, \quad N = \mathbf{P}_{tt} \cdot \hat{\mathbf{n}}. \quad (36)$$

Mean curvature H , Gaussian curvature K , and total curvature magnitude κ :

$$H = \frac{EN + GL - 2FM}{2(EG - F^2)}, \quad K = \frac{LN - M^2}{EG - F^2}, \quad \kappa = \sqrt{\max(4H^2 - 2K, 0)} \quad (= \sqrt{k_1^2 + k_2^2}). \quad (37)$$

6.3 Curvature monitor field

Per-cell curvature is clipped at its own 99.5th percentile (outlier robustness), normalized by the block’s own median κ_{ref} , scaled by an aggressiveness parameter β , and capped:

$$m_{\text{cell}} = \min\left(1 + \beta \frac{\kappa_{\text{cell}}}{\kappa_{\text{ref}}}, \quad m_{\text{max}}\right), \quad (38)$$

then averaged onto nodes over each node’s up-to-4 surrounding cells. $\beta = 0 \Rightarrow m \equiv 1$ (recovers plain, uniform elliptic smoothing).

6.4 Curvature-weighted 1D equidistribution

Redistributes a curve’s own points to equal increments of *weighted* arc length. With chord-length parameter s , total length S , and monitor $m(s)$ (linearly interpolated from a fine resampling):

$$W(s) = \int_0^s m(\sigma) d\sigma, \quad s_i = W^{-1}\left(\frac{i}{n-1} W(S)\right), \quad i = 0, \dots, n-1, \quad (39)$$

monotonic by construction; $m \equiv 1$ reproduces uniform arc-length spacing exactly.

6.5 Weighted elliptic relaxation + Newton reprojection

Each interior point is damped-relaxed toward its curvature-weighted 4-neighbor average (the discrete equilibrium of a spring network with local stiffness set by curvature), then reprojected onto the true surface via local Gauss–Newton refinement of its own (s, t) location:

$$\mathbf{r} = \mathbf{x}_{\text{target}} - \mathbf{P}(s, t), \quad (40)$$

$$\mathbf{J} = [\mathbf{P}_s, \mathbf{P}_t] \quad (\text{a } 3 \times 2 \text{ matrix}) \quad (\text{central-difference Jacobian}) \quad (41)$$

$$\delta = (\mathbf{J}^\top \mathbf{J} + \eta \mathbf{I})^{-1} \mathbf{J}^\top \mathbf{r}, \quad \eta = 10^{-12} \quad (\text{regularized normal equations}) \quad (42)$$

$$(s, t) \leftarrow (s, t) + \delta. \quad (43)$$

s, t are allowed to run outside $[0, 1]$ mid-solve; an overflow converts into an exact cell-index shift (cells are C^0 -continuous at $s = 1/t = 1$), and the Newton solve resumes in the new cell.

7 etaGrid_to_SOD2D/

See [docs/etaGrid_to_SOD2D/README.md](#). The underlying η -grid concept – wall-normal/spanwise grid sizes proportional to the local Kolmogorov scale η – is from Rouhi, Kumar, Wu, Kozul & Lehmkuhl, “Leveraging unstructured grids for direct numerical simulations of wall turbulence,” under review, *Journal of Fluid Mechanics* ([arXiv:2605.01015](#)); what follows is this repository’s own mesh-generation and SOD2D-import math built around that concept.

7.1 Airfoil curve: fit and differential geometry

Arc-length parametrization as in Section 2.4; a full cubic spline $\mathbf{P}(s) = (\text{spline}_x(s), \text{spline}_y(s))$ (`scipy.interpolate.CubicSpline`, not-a-knot end conditions). Tangent and outward normal:

$$\hat{\mathbf{t}}(s) = \frac{\mathbf{P}(s + \varepsilon) - \mathbf{P}(s - \varepsilon)}{2\varepsilon} / \|\cdot\|, \quad \hat{\mathbf{n}}(s) = (t_y(s), -t_x(s)) \quad (\text{tangent rotated } -90^\circ). \quad (44)$$

7.2 Vinokur (1983) two-sided stretching

Redistributes n points along a segment with independently specified first/last spacing $\delta s_0, \delta s_1$ (as fractions of the segment’s own total length; $s_0 = \delta s_0(n-1)$, $s_1 = \delta s_1(n-1)$):

$$A = \sqrt{s_1/s_0}, \quad B = \frac{1}{\sqrt{s_0 s_1}}. \quad (45)$$

Case $B > 1$ (spacings smaller than uniform): solve $\sinh(\Delta y)/\Delta y = B$ for Δy by Newton’s method (seeded by a small- $(B-1)$ series expansion, or $\log(2B) + \log \log(2B)$ for large B); then, with $\xi = i/(n-1)$,

$$u(\xi) = \frac{1}{2} \left(1 + \frac{\tanh(\Delta y(\xi - \frac{1}{2}))}{\tanh(\Delta y/2)} \right). \quad (46)$$

Case $B < 1$: the same construction with $\sin/\tan$ in place of $\sinh/\tanh$, solving $\sin(\Delta y)/\Delta y = B$.

Case $B \approx 1$: $u(\xi) = \xi$ (uniform). Finally,

$$t = \frac{u}{A + (1 - A)u}, \quad t_0 = 0, \quad t_{n-1} = 1 \text{ (exact)}. \quad (47)$$

Points are evaluated at $t \cdot (\text{total length})$ through a cubic-spline fit of the *original* segment, so resampled points still lie exactly on the input curve.

7.3 Body-wrap sweep geometry

Given a YZ cross-section local coordinate (Y_j, Z_j) at arc-length station s_k :

$$X(k, j) = P_x(s_k) + Y_j n_x(s_k), \quad (48)$$

$$Y(k, j) = P_y(s_k) + Y_j n_y(s_k), \quad (49)$$

$$Z(k, j) = Z_j, \quad (50)$$

i.e. a ruled-surface-style offset sweep: at $Y_j = 0$ every station lands exactly on the true curve; away from the wall, each node is carried at a fixed offset *along the local normal at that station* – contrast with Section 2’s pure-translation extrusion.

Orientation. A swept hex’s signed volume,

$$V = (\mathbf{e}_1 \times \mathbf{e}_2) \cdot \mathbf{e}_3, \quad \mathbf{e}_1 = \mathbf{p}_1 - \mathbf{p}_0, \quad \mathbf{e}_2 = \mathbf{p}_3 - \mathbf{p}_0, \quad \mathbf{e}_3 = \mathbf{p}_4 - \mathbf{p}_0, \quad (51)$$

can come out uniformly negative across an entire swept region (handedness ambiguity between the cross-section’s own 2D winding and the local sweep direction); corrected by a node-order swap $[4, 5, 6, 7, 0, 1, 2, 3]$ when the majority of a sampled batch is negative.

7.4 Wake growth: geometric offset series

Downstream station offsets $x_0 = 0, x_1 = s_0, x_2 = s_0(1 + r), \dots$ reach total in exactly n layers, where the ratio r solves

$$f(r) = s_0 \frac{r^n - 1}{r - 1} = \text{total} \quad (f(r) \rightarrow s_0 n \text{ as } r \rightarrow 1), \quad (52)$$

via bisection over $r \in [1, \infty)$ (no closed form for general n).

7.5 Boundary classification (local coordinates)

$$\text{wall} : |Y - Y_{\min}| < \text{tol}, \quad \text{outer} : |Y - Y_{\max}| < \text{tol}, \quad \text{periodic}_{0,1} : |Z - Z_{\min, \max}| < \text{tol}, \quad (53)$$

applied identically at every swept station since it depends only on the cross-section’s own local (Y, Z) , never on 3D position.

7.6 Wavy-wall perturbation

A second, independent displacement applied on top of an already spline-smoothed wall, along the local outward normal:

$$\Delta(s, z) = A_{\text{frac}} \cdot c \cdot \text{tent}\left(n_w \frac{z - z_{\min}}{z_{\text{span}}}\right) \cdot \tau(s), \quad (54)$$

where c is the chord (auto-computed as the spline table’s own X -extent), A_{frac} an amplitude fraction, n_w an integer wavenumber.

Chordwise taper.

$$\tau(s) = \sin^2\left(\pi \frac{s}{s_{\text{total}}}\right), \quad (55)$$

zero *with zero slope* at both $s = 0$ and $s = s_{\text{total}}$ (both ends of the arc-length parametrization are the trailing edge), peaking at the leading edge $s = s_{\text{total}}/2$.

Spanwise unipolar tent wave. With phase $\varphi = n_w(z - z_{\min})/z_{\text{span}} \bmod 1$:

$$\text{tent}(\varphi) = 1 - |2\varphi - 1|, \quad \varphi \in [0, 1], \quad (56)$$

range $[0, 1]$: zero (the *original*, undisplaced surface – the trough) at $\varphi = 0, 1$; peak 1 at $\varphi = 0.5$. Because n_w is an integer, φ at $z = z_{\max}$ is exactly an integer, so the wave is *exactly* zero at both spanwise-periodic boundaries.

Local tangent (Fortran evaluation). Reusing the cubic segment coefficients of Section 2.4:

$$t_x = b_x + \delta t (2c_x + 3\delta t d_x), \quad t_y = b_y + \delta t (2c_y + 3\delta t d_y), \quad \hat{\mathbf{n}} = \frac{(t_y, -t_x)}{\|(t_x, t_y)\|}. \quad (57)$$

The full imposed displacement is then $\mathbf{u}_{\text{buffer}} = \Delta(s, z) \hat{\mathbf{n}}$, with $u_{\text{buffer},z} \equiv 0$ always (displacement confined to the local X – Y plane at every Z).

7.7 Wall-node arc-length recovery (verification)

For an already-placed wall node, recover s via Newton/Gauss–Newton search minimizing $\|\mathbf{P}(s) - \mathbf{x}\|^2$:

$$f(s) = (\mathbf{P}(s) - \mathbf{x}) \cdot \hat{\mathbf{t}}(s), \quad f'(s) \approx \hat{\mathbf{t}}(s) \cdot \hat{\mathbf{t}}(s) \quad (\text{Gauss–Newton}), \quad s \leftarrow s - f(s)/f'(s), \quad (58)$$

seeded from the nearest of the curve’s own control points; converges in a handful of steps given a point already close to the curve.

8 Cross-cutting: boundary conditions

See [docs/reference/boundary-conditions.md](#) for the full physical-id table and `bc_type` string reference (not reproduced here as it is not mathematical content); the inlet/outlet classification math is Section 2.3 above, shared verbatim by every Gmsh-based pipeline that needs it.